

One material. One source of truth. Every consumer.

The complete flow from an immutable USDZ package in object storage to a verified, mounted texture set inside Autodesk Revit.



01 · Author

Ingest and validate material inputs

02 · Canonicalise

Build one immutable USDZ revision

03 · Project

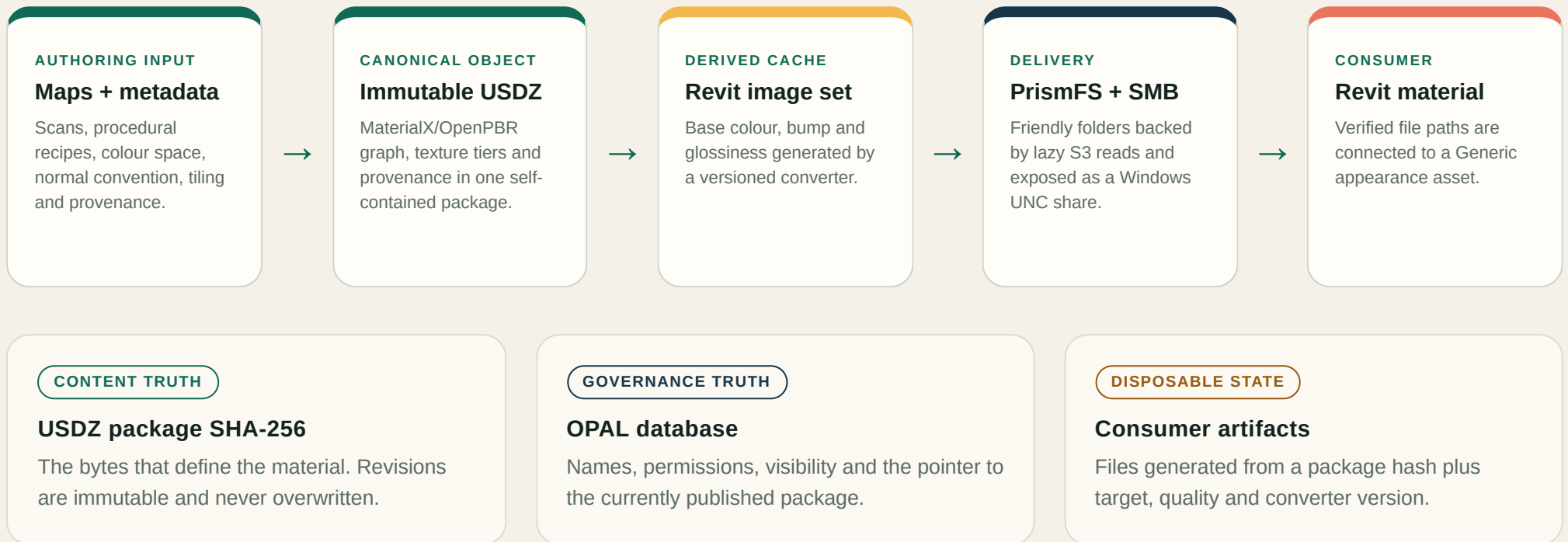
Generate disposable consumer caches

04 · Consume

Mount, verify and apply in Revit

The USDZ is canonical; everything else is a projection

OPAL separates durable material meaning from temporary delivery formats. A published version pins a package hash. Consumer outputs are reproducible caches that may be deleted and rebuilt.



How a material becomes publishable

Durable lane

SOURCE OF TRUTH

1. Ingest texture maps or bake a procedural definition.
2. Record explicit role, colour space, normal convention and physical scale.
3. Review the canonical authoring inputs.
4. Build a deterministic USDZ with the USD Toolbox.
5. Validate the package and calculate its SHA-256.
6. Store it at an immutable revision key in S3.
7. Pin that package in the material version.



Projection lane

REBUILDABLE

1. Request a target and quality, such as Revit 2K.
2. Look up the cache key for the package and converter.
3. Run `usd-toolbox export --target revit` on a miss.
4. Verify the exported manifest and each output hash.
5. Store outputs as content-addressed cache objects.
6. Project only ready artifacts into drive manifests.
7. Garbage-collect artifacts no published package references.

`artifact key = package SHA-256 + target + quality + converter + converter version`

Changing the material

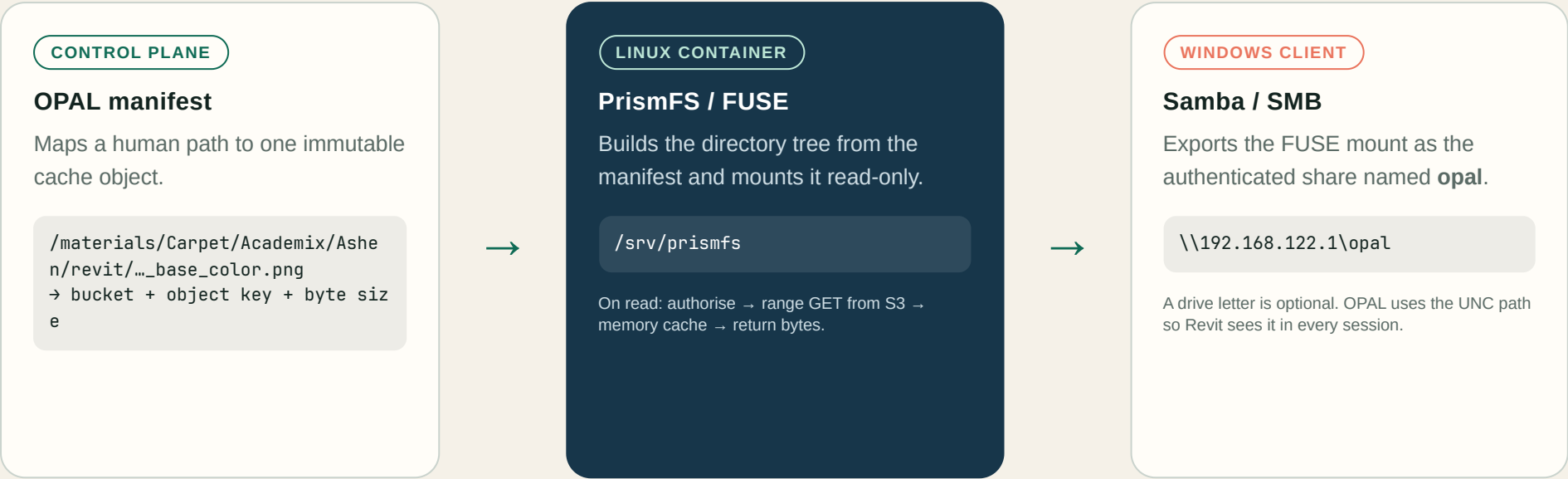
Creates a new USDZ revision and hash. Old versions continue resolving to their original bytes and caches.

Changing a converter

Creates a new cache generation without changing or repackaging the source material.

The “mounted drive” is a live, read-only view—not a bucket sync

The drive carries filenames and mappings up front. Actual texture bytes move only when Windows or Revit opens a file.



CREDENTIAL	PURPOSE	BOUNDARY
Drive bearer token	Fetches the protected namespace manifest and submits access events.	PrismFS ↔ OPAL API
S3 access key	Reads the immutable cache objects named by the manifest.	PrismFS ↔ object storage
SMB user/password	Lets Windows authenticate to the read-only opa ^l share.	Windows ↔ Samba
OPAL account token	Searches materials, resolves paths and registers Revit identity.	Revit add-in ↔ OPAL API

From “Apply Material” to pixels in the viewport

1

Select

The user selects a Revit material or clicks a face, then chooses an OPAL variant.

2

Resolve

The add-in asks OPAL for the Revit cache derived from the USDZ pinned by the current version, at the requested or best available quality.

3

Join paths

API paths are joined to `\\192.168.122.1\opal`.

4

Verify bytes

Each file is opened through SMB and its SHA-256 is compared with OPAL metadata.

5

Apply

A Revit transaction connects Unified Bitmap assets for base colour, bump and glossiness.

6

Scale and identify

Physical tile size, repeat settings and OPAL identity parameters are written.

7

Audit

OPAL records the Revit identity while PrismFS batches open/read events back to the control plane.

WHAT REVIT STORES

Paths, not the USDZ

The connector writes UNC texture paths into a Generic appearance asset. Revit does not currently import MaterialX or USDZ directly.

```
...\\revit\\2k\\CPT-TARKETT-ACADEMIX-ASHEN_base_color.png
```

First application

Hash verification reads the complete texture. PrismFS streams the data from S3 and caches ranges in memory.

Later rendering

Revit reopens the configured paths when it needs image data, so the UNC share remains the authoritative delivery path.

Properties the design guarantees

published material → exactly one package revision per variant → zero or more reproducible projections

- ✓

No drift. A drive artifact records the exact package hash and converter version that created it.
- ✓

Safe rollback. Moving the current-version pointer restores the prior package and matching caches.
- ✓

Immutable history. Packages and content-addressed outputs never change underneath a reference.
- ✓

Atomic publication. A package is exposed only after required consumer artifacts are complete.
- ✓

Rebuildable delivery. Deleting a cache never destroys material meaning; the USDZ recreates it.
- ✓

Lazy movement. A mount loads metadata first and transfers S3 bytes only when a file is read.

FAILURE	VISIBLE RESULT	RECOVERY
USDZ build or validation fails	No new package revision can be published.	Correct the authoring input; the prior package stays current.
Consumer export fails	The candidate package remains unexposed to that target.	Retry with the same package and converter, or ship a corrected converter version.
PrismFS/Samba is down	Search still works, but Revit reports that the drive is unavailable.	Restart the drive service; no material data needs rebuilding.
S3 is temporarily unavailable	Folder listings may work, while uncached file reads fail.	Restore object-store access; immutable object references remain valid.
Hash mismatch	The Revit connector refuses to apply the material.	Regenerate the derivative and investigate storage corruption.

The design principle

Store material meaning once. Convert for consumers deterministically. Cache for speed. Verify at every boundary.